

WOXSEN UNIVERSITY

DBMS Course Project Report

Design and Implementation of a Database Management System for Water Tanker Booking and Delivery Management System

Item	Details
Course	Database Management Systems (DBMS)
Submitted by	T Chandrahas
Roll number	25WU0102284
Programme and section	B.Tech (AIML), 2nd Year, Semester III, AIML Whales
University	Woxsen University
Faculty	Dr. Kiranmayee Adavala
Academic year	2026-2027
Date of submission	07.10.2026

1. Abstract

This project delivers a normalized MySQL database and a working web application for managing water tanker bookings and deliveries. Water tanker supply is still booked mostly by phone and paper registers, which causes lost orders, double-booked vehicles, blind dispatching and disputed cash payments. The database, named `water_tanker_db`, has nine tables in Third Normal Form (CUSTOMER, AREA, ADDRESS, TANKER_TYPE, TANKER, DRIVER, BOOKING, DELIVERY and PAYMENT), one reporting view (`vw_booking_details`) and three triggers. The triggers enforce the business rules inside the database: a tanker cannot be double-booked, its size must match the booking, a driver cannot be on two deliveries at once, and booking and tanker statuses update automatically. A React web application named AquaFlow sits on top of the database and supports inserting, deleting and viewing records, dispatching tankers, and tracking revenue. This report documents the problem, the ER model, the schema and normalization, the data dictionary, the SQL used, the queries, the user interface, the implementation and the testing.

2. Introduction and Problem Statement

2.1 Introduction

A water tanker service takes booking requests from customers for a delivery address and a time slot, assigns a tanker of the right size and a driver, delivers the water, and collects payment. Each step creates data that must stay consistent. A DBMS gives that consistency through tables, keys, constraints, triggers and transactions, and lets an operator answer questions with SQL.

2.2 Problem Statement

Manual water delivery suffers from four problems, identified in Presentation-I:

- Phone and paper bookings: orders are lost, copied wrongly, and there is no single record of demand.
- No inventory tracking: tanker availability is not tracked centrally, so vehicles get double-booked.
- Blind dispatching: without location data, dispatch is inefficient and customers get no reliable arrival estimate.
- Cash-only ledgers: paper payment records cause disputes and make auditing difficult.

The task is to design a normalized relational database and a user interface that record customers, addresses, tankers, drivers, bookings, deliveries and payments reliably, and that block invalid operations such as double-booking at the database layer.

3. Objectives and Scope

3.1 Objectives

- Model the water tanker domain with an Entity-Relationship (ER) diagram in Chen notation.
- Convert the model into a relational schema in Third Normal Form (3NF).
- Implement the schema in MySQL with primary keys, foreign keys, UNIQUE and CHECK constraints.
- Enforce dispatch rules with triggers so they hold whichever frontend is used.

- Build a web UI connected to MySQL that supports insert, delete and view operations.
- Demonstrate joins, aggregates, subqueries, a view, integrity checks and concurrency control.

3.2 Scope

In scope: customers with several delivery addresses, service areas, tanker sizes with prices, the tanker fleet, drivers, time-slot bookings, deliveries, payments, status automation, and revenue analytics.

Out of scope: real GPS hardware (the live radar in the UI is a simulation), online payment gateway integration, and user login with roles.

4. Software and Hardware Requirements

4.1 Software Requirements

Component	Details
Operating system	Windows 10/11
Database	MySQL Community Server (script written for 8.0+, demonstrated in the MySQL 9.7 command-line client)
Database tools	MySQL command-line client, MySQL Workbench
Runtime	Node.js (LTS) with npm
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Motion, Lucide icons
Server layer	Express with dotenv
IDE	PyCharm
Diagramming	draw.io (diagrams.net)
Version control	Git and GitHub
Browser	Google Chrome or Microsoft Edge

4.2 Hardware Requirements

Component	Minimum
Processor	Dual-core, 2 GHz or above
RAM	8 GB
Storage	2 GB free space
Display	1366 x 768 or higher

5. ER Diagram

The ER diagram (Chen notation) has nine entities: CUSTOMER, AREA, ADDRESS, TANKER_TYPE, TANKER, DRIVER, BOOKING, DELIVERY and PAYMENT.

[Insert Figure 1 here: the ER diagram image, water_tanker_er (Chen notation).png. Caption: ER diagram of the Water Tanker Booking and Delivery Management System]

5.1 Relationships

Relationship	Entities	Cardinality	Meaning
has	CUSTOMER - ADDRESS	1 : N	A customer can register many delivery addresses.
contains	AREA - ADDRESS	1 : N	An area holds many addresses; each address is in one area.
delivered to	ADDRESS - BOOKING	1 : N	An address can receive many bookings.
requested as	TANKER_TYPE - BOOKING	1 : N	A booking asks for one tanker size; a size is requested by many bookings.
classifies	TANKER_TYPE - TANKER	1 : N	One size category covers many tankers.
fulfilled by	BOOKING - DELIVERY	1 : 1	A booking has at most one delivery.
used in	TANKER - DELIVERY	1 : N	A tanker makes many deliveries over time.
drives	DRIVER - DELIVERY	1 : N	A driver makes many deliveries over time.
settled by	BOOKING - PAYMENT	1 : 1	One payment per booking.

6. Relational Schema and Normalization

6.1 Relational Schema

PK = primary key, FK = foreign key.

- CUSTOMER (CustomerID PK, Name, Phone)
- AREA (AreaID PK, AreaName, Pincode)
- ADDRESS (AddressID PK, CustomerID FK, AreaID FK, Street, Latitude, Longitude)
- TANKER_TYPE (TypeID PK, Capacity_Liters, Price)
- TANKER (TankerID PK, TypeID FK, License_Plate, Status)
- DRIVER (DriverID PK, Name, Phone, License_No)
- BOOKING (BookingID PK, AddressID FK, TypeID FK, ScheduledDate, TimeSlot, Status)
- DELIVERY (DeliveryID PK, BookingID FK, TankerID FK, DriverID FK, DispatchTime, DeliveredTime)
- PAYMENT (PaymentID PK, BookingID FK, Amount, Method)

6.2 Normalization

Normalization takes the paper booking register through Unnormalized Form (UNF), First Normal Form (1NF), Second Normal Form (2NF) and Third Normal Form (3NF). Every sample row below is a real row from water_tanker_db (bookings 1, 2, 4 and 9), so each table can be checked against the SQL output in Section 8. Bookings 1, 2 and 4 are delivered and paid. Booking 9 is Pending, so it has no tanker, driver or payment yet.

6.2.1 Unnormalized form (UNF)

A paper register writes each booking as one line, and several facts are packed into a single cell. Table 6.1 shows four bookings, one column per booking.

Table 6.1: Unnormalized booking register

Field	Booking 1	Booking 2	Booking 4	Booking 9
Customer	Ravi Kumar, 9000000001	Sneha Reddy, 9000000002	Lakshmi Devi, 9000000004	Ravi Kumar, 9000000001
Delivery address	12-3-45, Main Road, Gandhi Nagar - 500101	4-18, Temple Street, Lakshmi Nagar - 500102	9-1-7, Canal Road, Gandhi Nagar - 500101	12-3-45, Main Road, Gandhi Nagar - 500101
Tanker size and price	6000 L, 1000.00	10000 L, 1500.00	6000 L, 1000.00	3000 L, 600.00
Vehicle and driver	TS01WT1003; Ramesh Goud, DL-2026-0001	TS01WT1005; Venkatesh Yadav, DL- 2026-0002	TS01WT1004; Ramesh Goud, DL-2026-0001	not assigned yet
Schedule	2026-09-16, 06:00- 09:00	2026-09-16, 09:00- 12:00	2026-09-18, 09:00- 12:00	2026-09-22, 06:00- 09:00
Payment	1000.00 by UPI	1500.00 by Cash	1000.00 by Card	not paid yet

The cells are not atomic. Name and phone share one cell, and street, area and pincode share another. A query such as "all bookings in pincode 500101" cannot be answered without splitting the text.

6.2.2 First Normal Form (1NF)

A table is in 1NF when every column holds one atomic value, there are no repeating groups, and every row has a primary key. Splitting the composite cells and adding BookingID as the key gives the flat table:

BOOKING_ALL (BookingID PK, CustomerName, CustomerPhone, Street, AreaName, Pincode, Capacity_Liters, Price, LicensePlate, DriverName, DriverLicense, ScheduledDate, TimeSlot, Amount, Method)

Table 6.2: BOOKING_ALL in 1NF (shown transposed so it fits the page: each column is one row of the table)

Attribute	Booking 1	Booking 2	Booking 4	Booking 9
BookingID (PK)	1	2	4	9
CustomerName	Ravi Kumar	Sneha Reddy	Lakshmi Devi	Ravi Kumar
CustomerPhone	9000000001	9000000002	9000000004	9000000001
Street	12-3-45, Main Road	4-18, Temple Street	9-1-7, Canal Road	12-3-45, Main Road
AreaName	Gandhi Nagar	Lakshmi Nagar	Gandhi Nagar	Gandhi Nagar
Pincode	500101	500102	500101	500101
Capacity_Liters	6000	10000	6000	3000
Price	1000.00	1500.00	1000.00	600.00
LicensePlate	TS01WT1003	TS01WT1005	TS01WT1004	NULL
DriverName	Ramesh Goud	Venkatesh Yadav	Ramesh Goud	NULL
DriverLicense	DL-2026-0001	DL-2026-0002	DL-2026-0001	NULL
ScheduledDate	2026-09-16	2026-09-16	2026-09-18	2026-09-22
TimeSlot	06:00-09:00	09:00-12:00	09:00-12:00	06:00-09:00
Amount	1000.00	1500.00	1000.00	NULL
Method	UPI	Cash	Card	NULL

The table is in 1NF but still repeats data:

- Ravi Kumar's name and phone appear in bookings 1 and 9.
- Gandhi Nagar and pincode 500101 appear in bookings 1, 4 and 9.
- Ramesh Goud and DL-2026-0001 appear in bookings 1 and 4.
- The size 6000 L at 1000.00 appears in bookings 1 and 4.

This repetition causes the anomalies listed in Table 6.14.

6.2.3 Functional dependencies

Table 6.3 lists the functional dependencies (FDs) of the domain. They decide where each attribute belongs.

Table 6.3: Functional dependencies

Determinant	Determines	Meaning
BookingID	AddressID, TypeID, ScheduledDate, TimeSlot, Status	A booking fixes where, what size, and when
AddressID	CustomerID, AreaID, Street, Latitude, Longitude	An address belongs to one customer and one area
CustomerID	Name, Phone	Phone is also unique, so Phone determines CustomerID
AreaID	AreaName, Pincode	The pincode depends on the area
TypeID	Capacity_Liters, Price	The price depends on the tanker size
TankerID	TypeID, License_Plate, Status	A tanker has one size and one plate
DriverID	Name, Phone, License_No	A driver has one licence
DeliveryID	BookingID, TankerID, DriverID, DispatchTime, DeliveredTime	BookingID also determines DeliveryID (one delivery per booking)
PaymentID	BookingID, Amount, Method	BookingID also determines PaymentID (one payment per booking)

In BOOKING_ALL these FDs create transitive chains, where a non-key attribute depends on another non-key attribute:

- BookingID → customer → CustomerName, CustomerPhone
- BookingID → area → Pincode
- BookingID → tanker size → Price
- BookingID → tanker → LicensePlate
- BookingID → driver → DriverLicense

6.2.4 Second Normal Form (2NF)

A table is in 2NF when it is in 1NF and no non-key attribute depends on only part of a composite key.

BOOKING_ALL has the single-column key BookingID, so a partial dependency cannot occur and the table

is already in 2NF. The anomalies that remain come from the transitive chains above, which 3NF removes.

6.2.5 Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and no non-key attribute depends on another non-key attribute. Each transitive chain is cut by moving the dependent attributes into their own table, with a key that the original table keeps as a foreign key.

Table 6.4: Decomposition steps

Step	Dependency that breaks 3NF	New table
1	AreaName → Pincode	AREA (AreaID PK, AreaName, Pincode)
2	CustomerPhone → CustomerName	CUSTOMER (CustomerID PK, Name, Phone)
3	Street and area belong to a customer's address, not to the booking	ADDRESS (AddressID PK, CustomerID FK, AreaID FK, Street, Latitude, Longitude)
4	Capacity_Liters → Price	TANKER_TYPE (TypeID PK, Capacity_Liters, Price)
5	LicensePlate describes the tanker, not the booking	TANKER (TankerID PK, TypeID FK, License_Plate, Status)
6	DriverLicense → DriverName	DRIVER (DriverID PK, Name, Phone, License_No)
7	Tanker, driver and times describe the delivery event	DELIVERY (DeliveryID PK, BookingID FK, TankerID FK, DriverID FK, DispatchTime, DeliveredTime)
8	Amount and Method describe the payment	PAYMENT (PaymentID PK, BookingID FK, Amount, Method)
9	What is left is the customer's request	BOOKING (BookingID PK, AddressID FK, TypeID FK, ScheduledDate, TimeSlot, Status)

Notes on the decomposition:

- DELIVERY and PAYMENT are separate tables because a Pending booking has neither. This keeps NULL columns out of BOOKING.
- BOOKING keeps no CustomerID, because AddressID already leads to the customer.
- TANKER carries TypeID, so the delivery trigger can check that the tanker size matches the booking size.
- The flat sheet in Table 6.2 omits a few attributes (booking status, tanker status, driver phone, coordinates, dispatch and delivered times). They join the table they depend on.

The normalised tables with sample data

PK marks a primary key and FK a foreign key. The rows shown are those needed for bookings 1, 2, 4 and 9. The full tables hold more rows.

Table 6.5: AREA

AreaID (PK)	AreaName	Pincode
1	Gandhi Nagar	500101
2	Lakshmi Nagar	500102

Table 6.6: CUSTOMER

CustomerID (PK)	Name	Phone
1	Ravi Kumar	9000000001
2	Sneha Reddy	9000000002
4	Lakshmi Devi	9000000004

Table 6.7: ADDRESS

AddressID (PK)	CustomerID (FK)	AreaID (FK)	Street	Latitude	Longitude
1	1	1	12-3-45, Main Road	17.401200	78.480100
2	2	2	4-18, Temple Street	17.405800	78.487300
4	4	1	9-1-7, Canal Road	17.402600	78.478400

Table 6.8: TANKER_TYPE

TypeID (PK)	Capacity_Liters	Price
1	3000	600.00
2	6000	1000.00
3	10000	1500.00

Table 6.9: TANKER

TankerID (PK)	TypeID (FK)	License_Plate	Status
3	2	TS01WT1003	Dispatched
4	2	TS01WT1004	Available
5	3	TS01WT1005	Available

Table 6.10: DRIVER

DriverID (PK)	Name	Phone	License_No
1	Ramesh Goud	9100000001	DL-2026-0001
2	Venkatesh Yadav	9100000002	DL-2026-0002

Table 6.11: BOOKING

BookingID (PK)	AddressID (FK)	TypeID (FK)	ScheduledDate	TimeSlot	Status
1	1	2	2026-09-16	06:00-09:00	Delivered
2	2	3	2026-09-16	09:00-12:00	Delivered
4	4	2	2026-09-18	09:00-12:00	Delivered
9	1	1	2026-09-22	06:00-09:00	Pending

Table 6.12: DELIVERY (booking 9 has no row yet)

DeliveryID (PK)	BookingID (FK)	TankerID (FK)	DriverID (FK)	DispatchTime	DeliveredTime
1	1	3	1	2026-09-16 06:30:00	2026-09-16 07:40:00
2	2	5	2	2026-09-16 09:20:00	2026-09-16 10:35:00
4	4	4	1	2026-09-18 09:10:00	2026-09-18 10:20:00

Table 6.13: PAYMENT (booking 9 has no row yet)

PaymentID (PK)	BookingID (FK)	Amount	Method
1	1	1000.00	UPI
2	2	1500.00	Cash
4	4	1000.00	Card

6.2.6 Reconstructing the flat table (lossless join)

Joining the nine tables on their foreign keys gives back every row of Table 6.2, so no information was lost. The view vw_booking_details does this join:

```
SELECT BookingID, CustomerName, CustomerPhone, Street, AreaName, Pincode,
       Capacity_Liters, Price, License_Plate, DriverName,
       ScheduledDate, TimeSlot, PaidAmount, PaymentMethod
FROM vw_booking_details
WHERE BookingID IN (1, 2, 4, 9);
```

The result matches Table 6.2, apart from DriverLicense, which the view does not show.

[Paste a screenshot of this query's output here.]

6.2.7 Anomalies removed

Table 6.14: Anomalies in the flat table and how the normalised design handles them

Anomaly	In BOOKING_ALL	In the normalised design
Update	A new phone number for Ravi Kumar must be edited in bookings 1 and 9 and in every later booking. Missing one leaves contradictory data.	One UPDATE on one row of CUSTOMER.
Update	A corrected pincode for Gandhi Nagar must be edited in every booking in that area.	One UPDATE on one row of AREA.
Insertion	A new tanker, driver, area or tanker size cannot be stored until a booking uses it.	A plain INSERT into TANKER, DRIVER, AREA or TANKER_TYPE.
Deletion	Deleting a customer's last booking also erases the customer, the address and any tanker or driver seen only in that row.	DELETE FROM BOOKING removes only the booking. CUSTOMER, ADDRESS, TANKER and DRIVER stay, and ON DELETE RESTRICT blocks deleting a row that is still referenced.

6.2.8 Result

Every table is in 3NF. Each determinant in Table 6.3 is a candidate key of its own table, so the tables also satisfy Boyce-Codd Normal Form (BCNF).

Table 6.15: Summary of the normalised schema

Table	Primary key	Foreign keys	Other candidate keys (UNIQUE)	Normal form
CUSTOMER	CustomerID	none	Phone	3NF, BCNF
AREA	AreaID	none	AreaName	3NF, BCNF
ADDRESS	AddressID	CustomerID → CUSTOMER, AreaID → AREA	none	3NF, BCNF
TANKER_TYPE	TypeID	none	Capacity_Liters	3NF, BCNF
TANKER	TankerID	TypeID → TANKER_TYPE	License_Plate	3NF, BCNF
DRIVER	DriverID	none	License_No	3NF, BCNF
BOOKING	BookingID	AddressID → ADDRESS, TypeID → TANKER_TYPE	none	3NF, BCNF
DELIVERY	DeliveryID	BookingID → BOOKING, TankerID → TANKER, DriverID → DRIVER	BookingID	3NF, BCNF
PAYMENT	PaymentID	BookingID → BOOKING	BookingID	3NF, BCNF

In every table, each non-key attribute depends on the key, the whole key, and nothing but the key.

7. Data Dictionary

All tables use the InnoDB engine. Foreign keys use ON UPDATE CASCADE and ON DELETE RESTRICT.

7.1 CUSTOMER

Column	Data type	Constraints
CustomerID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(10)	NOT NULL, UNIQUE, CHECK (exactly 10 digits)

7.2 AREA

Column	Data type	Constraints
AreaID	INT	PRIMARY KEY, AUTO_INCREMENT
AreaName	VARCHAR(80)	NOT NULL, UNIQUE
Pincode	CHAR(6)	NOT NULL, CHECK (exactly 6 digits)

7.3 ADDRESS

Column	Data type	Constraints
AddressID	INT	PRIMARY KEY, AUTO_INCREMENT
CustomerID	INT	NOT NULL, FOREIGN KEY to CUSTOMER(CustomerID)
AreaID	INT	NOT NULL, FOREIGN KEY to AREA(AreaID)
Street	VARCHAR(150)	NOT NULL
Latitude	DECIMAL(9,6)	NULL, CHECK (-90 to 90)
Longitude	DECIMAL(9,6)	NULL, CHECK (-180 to 180)

7.4 TANKER_TYPE

Column	Data type	Constraints
TypeID	INT	PRIMARY KEY, AUTO_INCREMENT
Capacity_Liters	INT	NOT NULL, UNIQUE, CHECK (> 0)
Price	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)

7.5 TANKER

Column	Data type	Constraints
TankerID	INT	PRIMARY KEY, AUTO_INCREMENT
TypeID	INT	NOT NULL, FOREIGN KEY to TANKER_TYPE(TypeID)
License_Plate	VARCHAR(15)	NOT NULL, UNIQUE
Status	VARCHAR(15)	NOT NULL, DEFAULT 'Available', CHECK (Available, Dispatched, Maintenance)

7.6 DRIVER

Column	Data type	Constraints
DriverID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(10)	NOT NULL, CHECK (exactly 10 digits)
License_No	VARCHAR(20)	NOT NULL, UNIQUE

7.7 BOOKING

Column	Data type	Constraints
BookingID	INT	PRIMARY KEY, AUTO_INCREMENT
AddressID	INT	NOT NULL, FOREIGN KEY to ADDRESS(AddressID)

Column	Data type	Constraints
TypeID	INT	NOT NULL, FOREIGN KEY to TANKER_TYPE(TypeID)
ScheduledDate	DATE	NOT NULL
TimeSlot	VARCHAR(11)	NOT NULL, CHECK (06:00-09:00, 09:00-12:00, 12:00-15:00, 15:00-18:00)
Status	VARCHAR(10)	NOT NULL, DEFAULT 'Pending', CHECK (Pending, Assigned, Delivered, Cancelled)

An index, idx_booking_date_status, covers (ScheduledDate, Status).

7.8 DELIVERY

Column	Data type	Constraints
DeliveryID	INT	PRIMARY KEY, AUTO_INCREMENT
BookingID	INT	NOT NULL, UNIQUE, FOREIGN KEY to BOOKING(BookingID)
TankerID	INT	NOT NULL, FOREIGN KEY to TANKER(TankerID)
DriverID	INT	NOT NULL, FOREIGN KEY to DRIVER(DriverID)
DispatchTime	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
DeliveredTime	DATETIME	NULL, CHECK (NULL or not earlier than DispatchTime)

7.9 PAYMENT

Column	Data type	Constraints
PaymentID	INT	PRIMARY KEY, AUTO_INCREMENT
BookingID	INT	NOT NULL, UNIQUE, FOREIGN KEY to BOOKING(BookingID)
Amount	DECIMAL(10,2)	NOT NULL, CHECK (> 0)
Method	VARCHAR(12)	NOT NULL, CHECK (Cash, UPI, Card, NetBanking)

8. SQL Commands Used

The complete script is water_tanker_db.sql. It drops and recreates the database, so running it again resets the sample data.

8.1 DDL: Tables

Parent tables are created first and child tables after them.

```
DROP DATABASE IF EXISTS water_tanker_db;
CREATE DATABASE water_tanker_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE water_tanker_db;

CREATE TABLE CUSTOMER (
```

```

CustomerID INT AUTO_INCREMENT PRIMARY KEY,
Name VARCHAR(100) NOT NULL,
Phone VARCHAR(10) NOT NULL,
CONSTRAINT uq_customer_phone UNIQUE (Phone),
CONSTRAINT chk_customer_phone CHECK (Phone REGEXP '^[0-9]{10}$')
) ENGINE=InnoDB;

CREATE TABLE AREA (
AreaID INT AUTO_INCREMENT PRIMARY KEY,
AreaName VARCHAR(80) NOT NULL,
Pincode CHAR(6) NOT NULL,
CONSTRAINT uq_area_name UNIQUE (AreaName),
CONSTRAINT chk_area_pincode CHECK (Pincode REGEXP '^[0-9]{6}$')
) ENGINE=InnoDB;

CREATE TABLE ADDRESS (
AddressID INT AUTO_INCREMENT PRIMARY KEY,
CustomerID INT NOT NULL,
AreaID INT NOT NULL,
Street VARCHAR(150) NOT NULL,
Latitude DECIMAL(9,6) NULL,
Longitude DECIMAL(9,6) NULL,
CONSTRAINT fk_address_customer FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (CustomerID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_address_area FOREIGN KEY (AreaID) REFERENCES AREA (AreaID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_address_lat CHECK (Latitude BETWEEN -90 AND 90),
CONSTRAINT chk_address_lon CHECK (Longitude BETWEEN -180 AND 180)
) ENGINE=InnoDB;

CREATE TABLE TANKER_TYPE (
TypeID INT AUTO_INCREMENT PRIMARY KEY,
Capacity_Liters INT NOT NULL,
Price DECIMAL(10,2) NOT NULL,
CONSTRAINT uq_type_capacity UNIQUE (Capacity_Liters),
CONSTRAINT chk_type_capacity CHECK (Capacity_Liters > 0),
CONSTRAINT chk_type_price CHECK (Price >= 0)
) ENGINE=InnoDB;

CREATE TABLE TANKER (
TankerID INT AUTO_INCREMENT PRIMARY KEY,
TypeID INT NOT NULL,
License_Plate VARCHAR(15) NOT NULL,
Status VARCHAR(15) NOT NULL DEFAULT 'Available',
CONSTRAINT uq_tanker_plate UNIQUE (License_Plate),
CONSTRAINT fk_tanker_type FOREIGN KEY (TypeID) REFERENCES TANKER_TYPE (TypeID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_tanker_status CHECK (Status IN ('Available','Dispatched','Maintenance'))
) ENGINE=InnoDB;

CREATE TABLE DRIVER (
DriverID INT AUTO_INCREMENT PRIMARY KEY,
Name VARCHAR(100) NOT NULL,
Phone VARCHAR(10) NOT NULL,
License_No VARCHAR(20) NOT NULL,
CONSTRAINT uq_driver_license UNIQUE (License_No),
CONSTRAINT chk_driver_phone CHECK (Phone REGEXP '^[0-9]{10}$')
) ENGINE=InnoDB;

CREATE TABLE BOOKING (
BookingID INT AUTO_INCREMENT PRIMARY KEY,
AddressID INT NOT NULL,
TypeID INT NOT NULL,

```

```

ScheduledDate DATE NOT NULL,
TimeSlot VARCHAR(11) NOT NULL,
Status VARCHAR(10) NOT NULL DEFAULT 'Pending',
CONSTRAINT fk_booking_address FOREIGN KEY (AddressID) REFERENCES ADDRESS (AddressID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_booking_type FOREIGN KEY (TypeID) REFERENCES TANKER_TYPE (TypeID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_booking_slot CHECK (TimeSlot IN
('06:00-09:00', '09:00-12:00', '12:00-15:00', '15:00-18:00')),
CONSTRAINT chk_booking_status CHECK (Status IN
('Pending', 'Assigned', 'Delivered', 'Cancelled'))
) ENGINE=InnoDB;

CREATE INDEX idx_booking_date_status ON BOOKING (ScheduledDate, Status);

CREATE TABLE DELIVERY (
DeliveryID INT AUTO_INCREMENT PRIMARY KEY,
BookingID INT NOT NULL,
TankerID INT NOT NULL,
DriverID INT NOT NULL,
DispatchTime DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
DeliveredTime DATETIME NULL,
CONSTRAINT uq_delivery_booking UNIQUE (BookingID),
CONSTRAINT fk_delivery_booking FOREIGN KEY (BookingID) REFERENCES BOOKING (BookingID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_delivery_tanker FOREIGN KEY (TankerID) REFERENCES TANKER (TankerID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_delivery_driver FOREIGN KEY (DriverID) REFERENCES DRIVER (DriverID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_delivery_times CHECK (DeliveredTime IS NULL OR DeliveredTime >= DispatchTime)
) ENGINE=InnoDB;

CREATE TABLE PAYMENT (
PaymentID INT AUTO_INCREMENT PRIMARY KEY,
BookingID INT NOT NULL,
Amount DECIMAL(10,2) NOT NULL,
Method VARCHAR(12) NOT NULL,
CONSTRAINT uq_payment_booking UNIQUE (BookingID),
CONSTRAINT fk_payment_booking FOREIGN KEY (BookingID) REFERENCES BOOKING (BookingID)
ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT chk_payment_amount CHECK (Amount > 0),
CONSTRAINT chk_payment_method CHECK (Method IN ('Cash', 'UPI', 'Card', 'NetBanking'))
) ENGINE=InnoDB;

```

8.2 Triggers

Three triggers on DELIVERY keep the dispatch rules inside the database, so they hold no matter which application writes to it.

```

DELIMITER //

CREATE TRIGGER trg_delivery_before_insert
BEFORE INSERT ON DELIVERY FOR EACH ROW
BEGIN
    DECLARE v_booking_status VARCHAR(10);
    DECLARE v_booking_type INT;
    DECLARE v_tanker_status VARCHAR(15);
    DECLARE v_tanker_type INT;
    DECLARE v_driver_busy INT;

    SELECT Status, TypeID INTO v_booking_status, v_booking_type

```

```

    FROM BOOKING WHERE BookingID = NEW.BookingID;
SELECT Status, TypeID INTO v_tanker_status, v_tanker_type
    FROM TANKER WHERE TankerID = NEW.TankerID;
SELECT COUNT(*) INTO v_driver_busy
    FROM DELIVERY WHERE DriverID = NEW.DriverID AND DeliveredTime IS NULL;

IF v_booking_status IS NULL OR v_booking_status <> 'Pending' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Booking not found or not in Pending status';
END IF;
IF v_tanker_status IS NULL OR v_tanker_status <> 'Available' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tanker is not available';
END IF;
IF v_tanker_type <> v_booking_type THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tanker size does not match the booking';
END IF;
IF v_driver_busy > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver is already on another delivery';
END IF;
END//

CREATE TRIGGER trg_delivery_after_insert
AFTER INSERT ON DELIVERY FOR EACH ROW
BEGIN
    IF NEW.DeliveredTime IS NULL THEN
        UPDATE TANKER SET Status = 'Dispatched' WHERE TankerID = NEW.TankerID;
        UPDATE BOOKING SET Status = 'Assigned' WHERE BookingID = NEW.BookingID;
    ELSE
        UPDATE BOOKING SET Status = 'Delivered' WHERE BookingID = NEW.BookingID;
    END IF;
END//

CREATE TRIGGER trg_delivery_after_update
AFTER UPDATE ON DELIVERY FOR EACH ROW
BEGIN
    IF OLD.DeliveredTime IS NULL AND NEW.DeliveredTime IS NOT NULL THEN
        UPDATE BOOKING SET Status = 'Delivered' WHERE BookingID = NEW.BookingID;
        UPDATE TANKER SET Status = 'Available' WHERE TankerID = NEW.TankerID;
    END IF;
END//

DELIMITER ;

```

8.3 View

The view `vw_booking_details` returns one flat row per booking, joining customer, address, area, tanker size, delivery, tanker, driver and payment. The UI and the demo queries read from it.

```

CREATE VIEW vw_booking_details AS
SELECT b.BookingID, c.CustomerID, c.Name AS CustomerName, c.Phone AS CustomerPhone,
    a.Street, ar.AreaName, ar.Pincode, tt.Capacity_Liters, tt.Price,
    b.ScheduledDate, b.TimeSlot, b.Status AS BookingStatus,
    d.DeliveryID, t.License_Plate, dr.Name AS DriverName,
    d.DispatchTime, d.DeliveredTime,
    p.Amount AS PaidAmount, p.Method AS PaymentMethod
FROM BOOKING b
JOIN ADDRESS a ON a.AddressID = b.AddressID
JOIN CUSTOMER c ON c.CustomerID = a.CustomerID
JOIN AREA ar ON ar.AreaID = a.AreaID
JOIN TANKER_TYPE tt ON tt.TypeID = b.TypeID
LEFT JOIN DELIVERY d ON d.BookingID = b.BookingID
LEFT JOIN TANKER t ON t.TankerID = d.TankerID
LEFT JOIN DRIVER dr ON dr.DriverID = d.DriverID

```

```
LEFT JOIN PAYMENT p ON p.BookingID = b.BookingID;
```

8.4 DML: Sample Data

The script loads 6 areas, 8 customers, 10 addresses, 3 tanker sizes, 6 tankers, 4 drivers, 12 bookings, 8 deliveries and 7 payments. Parents are inserted before children. Representative statements:

```
INSERT INTO AREA (AreaName, Pincode) VALUES ('Gandhi Nagar', '500101'), ('Lakshmi Nagar', '500102');
INSERT INTO CUSTOMER (Name, Phone) VALUES ('Ravi Kumar', '9000000001'), ('Sneha Reddy', '9000000002');
INSERT INTO ADDRESS (CustomerID, AreaID, Street, Latitude, Longitude)
VALUES (1, 1, '12-3-45, Main Road', 17.401200, 78.480100);
INSERT INTO TANKER_TYPE (Capacity_Liters, Price) VALUES (3000, 600.00), (6000, 1000.00), (10000, 1500.00);
INSERT INTO TANKER (TypeID, License_Plate, Status) VALUES (1, 'TS01WT1001', 'Available');
INSERT INTO DRIVER (Name, Phone, License_No) VALUES ('Ramesh Goud', '9100000001', 'DL-2026-0001');
INSERT INTO BOOKING (AddressID, TypeID, ScheduledDate, TimeSlot, Status)
VALUES (1, 2, '2026-09-16', '06:00-09:00', 'Pending');
INSERT INTO DELIVERY (BookingID, TankerID, DriverID, DispatchTime, DeliveredTime)
VALUES (1, 3, 1, '2026-09-16 06:30:00', '2026-09-16 07:40:00');
INSERT INTO PAYMENT (BookingID, Amount, Method) VALUES (1, 1000.00, 'UPI');
```

Bookings are loaded as Pending. The delivery triggers then move them to Assigned or Delivered by themselves.

8.5 Sample Outputs

SHOW TABLES lists the nine tables and the view:

```
mysql> use water_tanker_db;
Database changed
mysql> show tables;
+-----+
| Tables_in_water_tanker_db |
+-----+
| address                    |
| area                      |
| booking                   |
| customer                  |
| delivery                  |
| driver                    |
| payment                   |
| tanker                    |
| tanker_type               |
| vw_booking_details        |
+-----+
10 rows in set
```

SELECT * FROM customer:

CustomerID	Name	Phone
1	Ravi Kumar	9000000001
2	Sneha Reddy	9000000002
3	Anil Sharma	9000000003
4	Lakshmi Devi	9000000004
5	Mohammed Imran	9000000005
6	Priya Nair	9000000006
7	Suresh Babu	9000000007

```
|          8 | Kavitha Rao | 9000000008 |
+-----+-----+-----+
8 rows in set
```

[Insert Figures 2 to 4 here: the MySQL command-line screenshots of SHOW TABLES, SELECT * FROM customer, and SELECT * FROM vw_booking_details.]

9. Queries with Outputs

All queries are in demo_queries.sql and run after water_tanker_db.sql. Output depends on the data at the time of the run, so paste a screenshot of your own run under each query.

9.1 Pending bookings per area (3-table join, GROUP BY)

```
SELECT ar.AreaName, COUNT(*) AS pending_bookings
FROM BOOKING b
JOIN ADDRESS a ON a.AddressID = b.AddressID
JOIN AREA ar ON ar.AreaID = a.AreaID
WHERE b.Status = 'Pending'
GROUP BY ar.AreaName
ORDER BY pending_bookings DESC;
```

[Output screenshot]

9.2 Every booking with customer, area and tanker size (5-table join)

```
SELECT b.BookingID, c.Name AS Customer, ar.AreaName, tt.Capacity_Liters,
       b.ScheduledDate, b.TimeSlot, b.Status
FROM BOOKING b
JOIN ADDRESS a ON a.AddressID = b.AddressID
JOIN CUSTOMER c ON c.CustomerID = a.CustomerID
JOIN AREA ar ON ar.AreaID = a.AreaID
JOIN TANKER_TYPE tt ON tt.TypeID = b.TypeID
ORDER BY b.ScheduledDate, b.TimeSlot;
```

[Output screenshot]

9.3 Revenue by tanker size (aggregate)

```
SELECT tt.Capacity_Liters, COUNT(p.PaymentID) AS payments, SUM(p.Amount) AS revenue
FROM PAYMENT p
JOIN BOOKING b ON b.BookingID = p.BookingID
JOIN TANKER_TYPE tt ON tt.TypeID = b.TypeID
GROUP BY tt.Capacity_Liters
ORDER BY revenue DESC;
```

[Output screenshot]

9.4 Trips per tanker, including tankers with none (LEFT JOIN)

```
SELECT t.License_Plate, t.Status, COUNT(d.DeliveryID) AS trips
FROM TANKER t
LEFT JOIN DELIVERY d ON d.TankerID = t.TankerID
GROUP BY t.TankerID, t.License_Plate, t.Status
ORDER BY trips DESC;
```

[Output screenshot]

9.5 Customers with more than one booking (GROUP BY and HAVING)

```
SELECT c.Name, COUNT(b.BookingID) AS total_bookings
FROM CUSTOMER c
JOIN ADDRESS a ON a.CustomerID = c.CustomerID
JOIN BOOKING b ON b.AddressID = a.AddressID
GROUP BY c.CustomerID, c.Name
HAVING COUNT(b.BookingID) > 1;
```

[Output screenshot]

9.6 Tankers that can serve pending booking 9 (subquery)

```
SELECT t.TankerID, t.License_Plate
FROM TANKER t
WHERE t.Status = 'Available'
AND t.TypeID = (SELECT TypeID FROM BOOKING WHERE BookingID = 9);
```

[Output screenshot]

9.7 Average delivery time per driver in minutes (date function)

```
SELECT dr.Name, COUNT(*) AS completed_trips,
       ROUND(AVG(TIMESTAMPDIFF(MINUTE, d.DispatchTime, d.DeliveredTime))) AS avg_minutes
FROM DELIVERY d
JOIN DRIVER dr ON dr.DriverID = d.DriverID
WHERE d.DeliveredTime IS NOT NULL
GROUP BY dr.DriverID, dr.Name;
```

[Output screenshot]

9.8 Deliveries currently on the road (uses the view)

```
SELECT BookingID, CustomerName, AreaName, License_Plate, DriverName, DispatchTime
FROM vw_booking_details
WHERE BookingStatus = 'Assigned';
```

[Output screenshot]

9.9 Trigger demonstration

Assigning a tanker and driver is one INSERT. The triggers change the booking and tanker statuses with no manual update.

```
-- Before: booking 9 is Pending, tanker 2 is Available
SELECT BookingID, Status FROM BOOKING WHERE BookingID = 9;
SELECT TankerID, Status FROM TANKER WHERE TankerID = 2;

-- Assign tanker 2 and driver 3 to booking 9
INSERT INTO DELIVERY (BookingID, TankerID, DriverID) VALUES (9, 2, 3);
-- After: booking 9 is Assigned, tanker 2 is Dispatched

-- Complete the delivery
UPDATE DELIVERY SET DeliveredTime = NOW() WHERE BookingID = 9;
-- Booking 9 is Delivered again, tanker 2 is Available again
```

[Output screenshots: the SELECTs before and after the INSERT and after the UPDATE]

9.10 Concurrency demonstration

Two query tabs prove that a tanker row is locked while one transaction works on it. Tab 2 waits until Tab 1 commits.

```
-- Tab 1
START TRANSACTION;
SELECT * FROM TANKER WHERE TankerID = 4 AND Status = 'Available' FOR UPDATE;
-- run the same SELECT ... FOR UPDATE in Tab 2: it waits
COMMIT; -- Tab 2 now continues
```

9.11 Query given during Presentation-II

[Paste the exact question you were given in Presentation-II, your SQL solution, and a screenshot of its output here. This section is required by the guidelines.]

10. UI Design and Screenshots

AquaFlow is a dark-themed web dashboard for the dispatcher, with a top bar of live counters (tankers in transit, available fleet) and a navigation row: Dashboard, Live GPS Radar, Active Deliveries, Fleet and Inventory, Bookings, Customers, Database Studio (9 tables) and Analytics. Every screen reads from MySQL, and every change is saved to MySQL and shown again after a refresh.

10.1 The three required operations

Operation	Where in the UI	What happens in the database
Insert	New Booking, Add New Customer, Dispatch Tanker	New rows in BOOKING, CUSTOMER and ADDRESS, or DELIVERY (the triggers then set Assigned and Dispatched)
Delete	Trash icon on each booking and customer row; Abort on an assigned booking	The row is removed; a delete that would break a foreign key is refused and the data stays intact
View	Dashboard, Bookings registry, Customer directory, Fleet and Inventory, Active Deliveries, Analytics, Database Studio	SELECT queries over the tables and the vw_booking_details view

Each booking and customer row also has an edit icon.

10.2 Screens

Dashboard. Four live cards: tankers in transit, tankers available at the depot out of six, pending dispatch orders, and total receipts (7,200 rupees in the captured run). Below them are the in-transit tanker with its driver, route progress and a Confirm Delivery button, and the pending orders queue with a Dispatch button and a delete icon on each order.

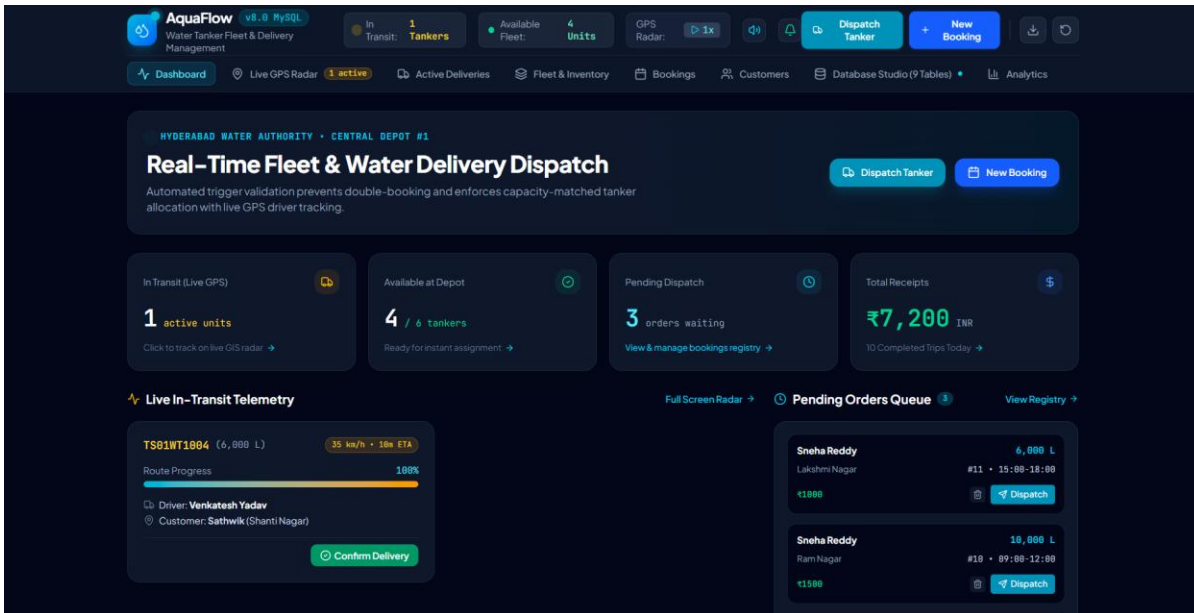


Figure 5: AquaFlow dashboard

Bookings registry. A table of every booking with booking ID, customer and phone, address and area with pincode, capacity and rate, date and slot, status, assigned tanker and driver, and row actions. A search box and Status and Time Slot filters narrow the list. The captured run shows bookings #13 to #15 created through the UI, with #15 in Assigned status on tanker TS01WT1004.

AquaFlow v6.8 MySQL Water Tanker Fleet & Delivery Management							
Water Tanker Bookings Registry							
Relational order lifecycle tracking: Pending → Assigned → Delivered / Cancelled.							
Search by customer name, phone, area, order ID...				Status: All Statuses	Slot: All Time Slots		
BOOKING ID	CUSTOMER	ADDRESS / AREA	CAPACITY & RATE	DATE & SLOT	STATUS	DELIVERY & DRIVER	ACTIONS
#15	Sathwik +91 8712254111	Flat 104 Shanti Nagar (PIN 500105)	6,000 L ₹1800	2026-10-08 09:00-12:00	Assigned	TS01WT1004 Driver: Venkatesh Yadav	Abort
#14	Sathwik +91 8712254111	Flat 104 Indra Colony (PIN 500104)	6,000 L ₹1800	2026-10-07 09:00-12:00	Delivered	TS01WT1003 Driver: Ramesh Goud	
#13	Sathwik +91 8790143199	shanmukh residency Indra Colony (PIN 500104)	6,000 L ₹1800	2026-10-07 09:00-12:00	Delivered	TS01WT1004 Driver: Srinivas Rao	
#1	Ravi Kumar +91 9888888881	12-3-45, Main Road Gandhi Nagar (PIN 500107)	6,000 L ₹1800	2026-09-16 06:00-09:00	Delivered	TS01WT1003 Driver: Ramesh Goud	
#2	Sneha Reddy +91 9888888882	4-18, Temple Street Lakshmi Nagar (PIN 500102)	10,000 L ₹1800	2026-09-16 09:00-12:00	Delivered	TS01WT1005 Driver: Venkatesh Yadav	
#3	Anil Sharma +91 9888888883	7-2-11, Market Lane Ram Nagar (PIN 500103)	3,000 L ₹400	2026-09-17 12:00-15:00	Delivered	TS01WT1001 Driver: Srinivas Rao	
#4	Lakshmi Devi +91 9888888884	9-1-7, Canal Road Gandhi Nagar (PIN 500107)	6,000 L ₹1800	2026-09-18 09:00-12:00	Delivered	TS01WT1004 Driver: Ramesh Goud	
#5	Mohammed Imran	2-44, School Street	6,000 L	2026-09-19	Delivered	TS01WT1003	

Figure 6: Bookings registry

Customer directory. Summary cards (11 customers, 15 bookings, 10 completed trips, 10,200 rupees of delivered inflow), a search box, and one row per customer with phone, registered addresses, booking count, a Book Tanker button, and edit and delete icons.

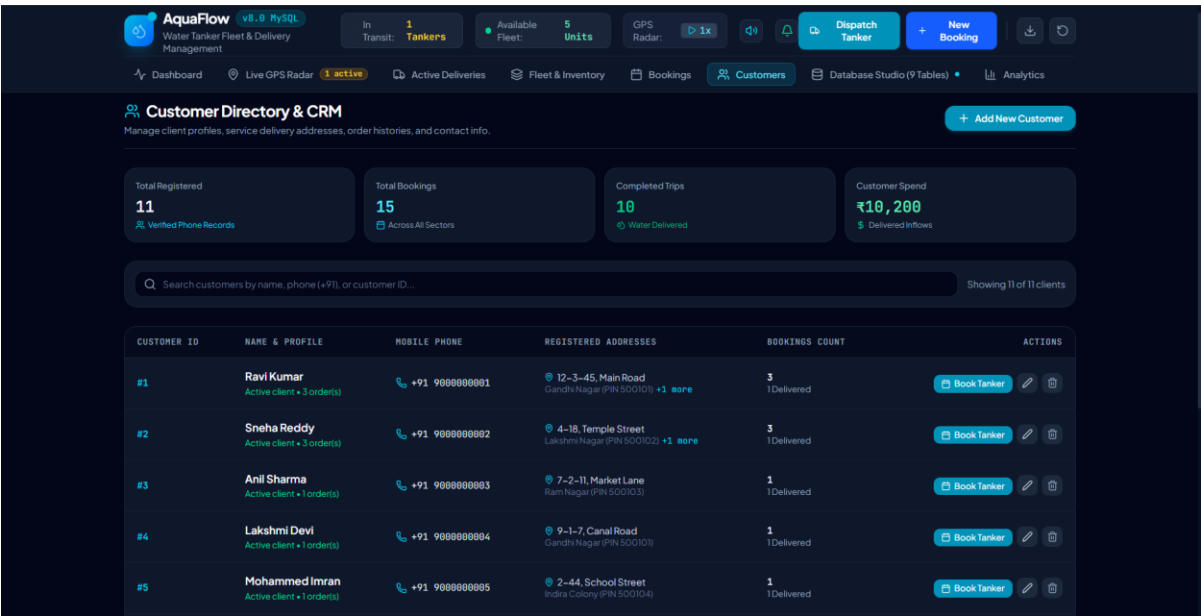


Figure 7: Customer directory

Analytics. A 30-day cumulative revenue curve with the average daily run-rate, the best single day and the number of active inflow days, with a switch to daily revenue inflow.

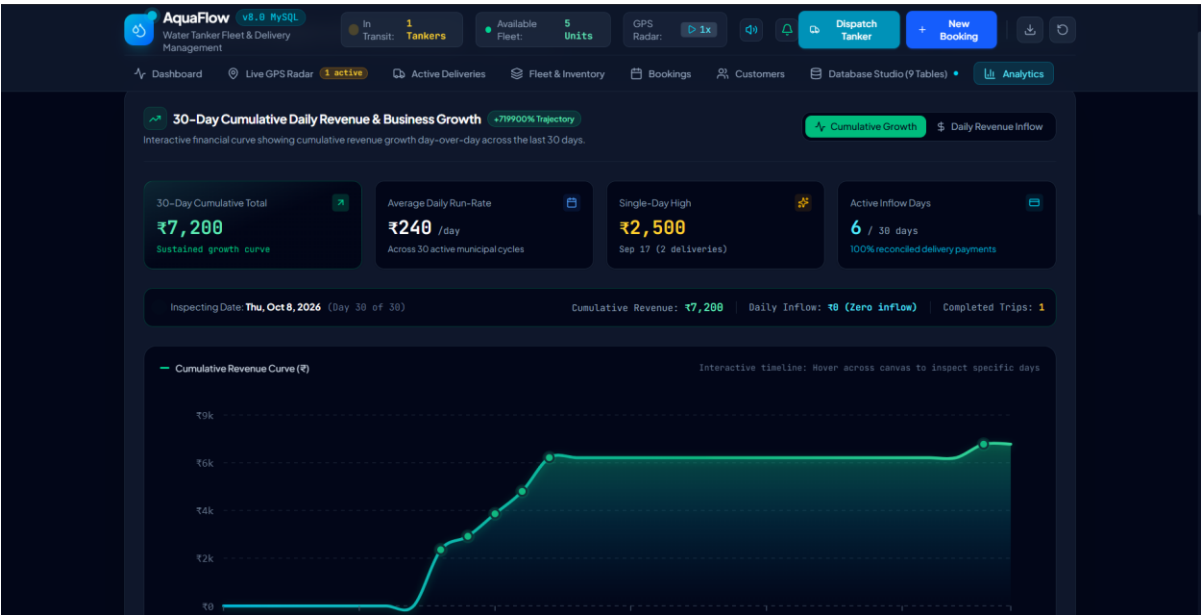


Figure 8: Analytics

[Insert before and after screenshots for the insert and delete operations here: a booking or customer before it is added or deleted, the screen after it, and the matching SELECT in MySQL.]

11. Implementation Details

11.1 Technology Stack

Layer	Technology
Database	MySQL, InnoDB engine, triggers, view, CHECK constraints
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Motion, Lucide icons
Server layer	Node.js with Express and dotenv
Tools	MySQL command-line client, MySQL Workbench, PyCharm, Git and GitHub

11.2 Architecture

A browser cannot talk to MySQL directly, so the React UI calls a small Express server, and the server runs SQL against MySQL:

React UI (browser) -> Express server (Node.js) -> MySQL database (water_tanker_db)

11.3 Where the Rules Live

The dispatch rules are in the database, not in the UI code. Dispatching a tanker is one statement:

```
INSERT INTO DELIVERY (BookingID, TankerID, DriverID) VALUES (9, 2, 3);
```

The BEFORE INSERT trigger rejects the statement with a clear message if the booking is not Pending, the tanker is not Available, the tanker size does not match the booking, or the driver is already on a delivery. The AFTER INSERT trigger then marks the booking Assigned and the tanker Dispatched. The AFTER UPDATE trigger closes the booking and frees the tanker when DeliveredTime is filled in. The UI shows the trigger's message when a dispatch is refused. This protects the data even if another program writes to the database.

Other integrity features: ON DELETE RESTRICT on every foreign key, UNIQUE on one delivery and one payment per booking, CHECK constraints on phone, pincode, coordinates, time slot, statuses, amounts and delivery times, and row locking with SELECT ... FOR UPDATE for concurrent dispatchers.

11.4 Database Connection Code

[Paste the connection code from your server file here: host, user, database name, and how the password is read from the .env file. Add one or two route handlers that show a view, an insert and a delete.]

12. Testing

The cases below come from the demo script (demo_queries.sql) and the UI. Fill in the Result column after each run, and attach a screenshot for the important ones.

ID	Test case	Action	Expected result	Result
T1	Load the schema	Run water_tanker_db.sql	9 tables and the view vw_booking_details exist	Pass (Figure 2)

ID	Test case	Action	Expected result	Result
T2	View data in SQL	SELECT * FROM customer	8 sample customers listed	Pass (Figure 3)
T3	View in the UI	Open the Customers and Bookings pages	Rows match the database	Pass (Figures 6, 7)
T4	Insert through the UI	Create a booking in New Booking	New row appears in BOOKING and in the registry	Pass (bookings #13 to #15, Figure 6)
T5	Dispatch	Assign a tanker and driver to a Pending booking	Booking becomes Assigned, tanker becomes Dispatched	Pass (booking #15, Figure 6)
T6	Complete a delivery	Confirm Delivery, or set DeliveredTime	Booking becomes Delivered, tanker becomes Available	[Pass/Fail]
T7	Delete through the UI	Delete a booking or customer with no dependent rows	Row removed from the screen and from MySQL	[Pass/Fail]
T8	Tanker in the workshop	INSERT INTO DELIVERY (10, 6, 3)	Rejected: Tanker is not available	[Pass/Fail]
T9	Double-booking	INSERT INTO DELIVERY (10, 1, 3) while tanker 1 is on the road	Rejected: Tanker is not available	[Pass/Fail]
T10	Wrong tanker size	INSERT INTO DELIVERY (10, 4, 3)	Rejected: Tanker size does not match the booking	[Pass/Fail]
T11	Busy driver	INSERT INTO DELIVERY (10, 5, 1) while driver 1 is on a delivery	Rejected: Driver is already on another delivery	[Pass/Fail]
T12	Booking already delivered	INSERT INTO DELIVERY (1, 2, 3)	Rejected: Booking not found or not in Pending status	[Pass/Fail]
T13	Foreign key	INSERT a booking with AddressID 999	Rejected by the foreign key	[Pass/Fail]
T14	CHECK constraint	INSERT a booking with time slot 23:00-24:00	Rejected by the CHECK constraint	[Pass/Fail]
T15	Referential integrity	DELETE the customer with ID 1	Blocked by ON DELETE RESTRICT	[Pass/Fail]
T16	UNIQUE constraint	Insert a second payment for booking 1	Rejected: one payment per booking	[Pass/Fail]
T17	Concurrency	Run SELECT ... FOR UPDATE on tanker 4 in two tabs	Tab 2 waits until Tab 1 commits	[Pass/Fail]

13. Conclusion and Future Enhancements

13.1 Conclusion

The project replaces phone calls and paper ledgers with a normalized MySQL database and the AquaFlow web application. The nine-table schema is in Third Normal Form, and the triggers, constraints and locking keep the data valid however it is changed. The UI supports insert, delete and view operations, tanker dispatch with automatic status changes, and revenue analytics, and every change is reflected in the database.

13.2 Future Enhancements

- Real GPS tracking from tanker devices, replacing the simulated live radar.
- Online payment gateway integration for UPI and card payments.

- Login with roles for admin, dispatcher and driver.
- Customer mobile app with SMS or WhatsApp booking alerts.
- Automatic tanker and driver suggestion by area and availability.
- Cloud deployment with scheduled database backups.

14. References

1. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill.
3. MySQL Reference Manual, <https://dev.mysql.com/doc/>
4. Node.js Documentation, <https://nodejs.org/en/docs>
5. Express Documentation, <https://expressjs.com>
6. React Documentation, <https://react.dev>
7. Tailwind CSS Documentation, <https://tailwindcss.com/docs>
8. draw.io (diagrams.net), <https://www.diagrams.net>
9. DBMS course lecture notes, Woxsen University.

15. Appendix: GitHub Repository Link

Repository: <https://github.com/CodeInfinity08/DBMS-Course-Project>

The repository holds the folders Presentation-I, Presentation-II, Presentation-III and Project-Report, with a README.md.